A Cup of Tea? Teapots, metadata and AI on Europeana

Author: Jenske Verhamme, Date: 01/05/2025

Table of Contents:

CHAPTER FOUR: IIIF AND GITHUB

Important note: All files used in this chapter are documented and downloadable on the Github page of this project!

link to project: https://github.com/GuacamoleKoala/EuropeanaTeapots/

overview of used datasets

1. initial dataset: Dataset_Initial_NotCleaned_EuropeanaTeapots.csv
2. extra dataset: DatasetEuropeana_SpecificMetadata.csv
3. dataset cleaned in OpenRefine: DatasetCleanedEuropeanaTeapots.openrefine.tar.gz
4. dataset in Tableau: TeapotDatasetEuropeanaTableau.csv

IIIF viewer

Thumbnails: grouped per 1000 images: 1-1000.rar, 1001-2000.rar, and so on. and logbook: image_download_log.csv

iiif manifest urls (.txt): iiif_manifest_urls.txt

iiif collection manifest url (json): IIIFCollectionURLS.json link: github.com/GuacamoleKoala/EuropeanaTeapots/IIIFViewer/IIIFCollectionURLS.json

script Mirador (html): CollectionThroughMiradorViewer.html

4.1 Image extraction and/or IIIF

I got the idea of wanting to use the images related to the teapots. I want to be able to make a website to show them or use a viewer to watch my collection.

This can also be interesting to clean the data in a different way as this visualisation will immediatly make clear to the naked eye what is a teapot and what is not. This also forms a pinnacle of this project, as we will finally be able to scroll through a digital collection of teapots from all around Europe (and the world).

In a future part I want to look more into IIIF and how to use it.

https://pro.europeana.eu/page/europeana-and-iiif?__cf_chl_tk=OrgAEC9QcJcfIdLu.002GWeOc.dI92enoC9lDSIBtIk-1746138695-1.0.1.1-XSNUVEXwmLQvg7kdm7MF_OSlK_vu_x38.4.s6GQbvDU

I asked Gemini 2,5 to look into the matter as well with this prompt:

> prompt: i want to use images from europeana to view in a iiif viewer. How do i do this? I have an api key. I also already wrote a script that extracts a dataset of teapots. I cleaned this dataset in excel and openrefine and have it visualised in tableau. Now I want to use the api to extract data about the images and how to view them in a viewer. Can you help me?

It responded with a plan to get the IIIF manifests of the records of my. This can be done by analyzing the JSON structure and retrieve the appropriate field. If we also extract the Europeana ID, we can then join the extracted data with our original dataset. We can also adjust our original script to also retrieve this field. In this case the difference does not matter much any longer, because we already almost finished with the cleaning and further processing of the data. At this point, we can just match the extracted data with our original dataset.

4.1.1 Thumbnail Images: Low Quality

An idea I had that can be a simple solution is to download all preview/ thumbnail images in a local folder via a script. Then I can watch them in a folder on my computer. In a later instance I can look up the HD image and watch those via separate records on Europeana.

First we upload our cleaned dataset to Google Colab:

In [ ]:
```
from google.colab import files
uploaded = files.upload()
```

Once the dataset is uploaded in Tableau, I generated a script via Gemini to download all thumbnails/preview images into a zip.

In [ ]:

```python
import pandas as pd
import requests
import os
from urllib.parse import urlparse
import hashlib # For creating a hash from the URL to ensure uniqueness
import time    # For adding delays if needed
import re      # For sanitizing filenames
import zipfile # For creating zip archives
from google.colab import files # Import for Colab-specific file operations

def download_images_from_dataframe(dataframe_path, image_url_column, output_
    """
    Downloads images from URLs specified in a DataFrame column, renames them
    as 'image_X.ext' upon successful download, and creates a log file mappir
    original URLs/records to final downloaded filenames.

    Args:
        dataframe_path (str): The path to your dataset file (e.g., 'teapots.
        image_url_column (str): The name of the column containing image URLs
        output_folder (str): The name of the folder where images will be sav
                             Defaults to 'downloaded_images'.
    """

    # --- 1. Create Output Folder ---
    if not os.path.exists(output_folder):
        os.makedirs(output_folder)
        print(f"Created output folder: '{output_folder}'")
    else:
        print(f"Output folder '{output_folder}' already exists.")

    # --- 2. Load the Dataset ---
    try:
        # Added dtype specification as suggested by the DtypeWarning from pr
        df = pd.read_csv(dataframe_path, dtype={'Column19': str, 'Column24':
        print(f"Successfully loaded dataset from: '{dataframe_path}'")
    except FileNotFoundError:
        print(f"Error: Dataset file not found at '{dataframe_path}'. Please
        return
    except Exception as e:
        print(f"Error loading dataset: {e}")
        return

    # --- 3. Initialize Log Data and Counters ---
    download_log = []
    download_count = 0  # Counts successfully downloaded files
    skipped_count = 0
    error_count = 0
    image_sequential_counter = 0 # Counter for "image_X" naming

    if image_url_column not in df.columns:
        print(f"Error: Column '{image_url_column}' not found in the dataset.
        print(f"Available columns are: {df.columns.tolist()}")
        return

    print("Starting image download process...")
```

```
    for index, row in df.iterrows():
        image_url = row[image_url_column]
        generated_filename = None # Will store the filename used for downloa
        status = "Processing"
        error_message = None
        ext = '.jpg' # Default extension, will be updated

        if pd.isna(image_url) or not isinstance(image_url, str) or not image
            status = "Skipped (Invalid URL)"
            skipped_count += 1
            error_message = "Empty or invalid URL found."
            download_log.append({
                'dataframe_index': index,
                'original_url': image_url,
                'generated_filename': None, # No filename generated
                'download_status': status,
                'error_message': error_message
            })
            continue

        try:
            # --- Initial Filename Generation (using your existing logic) --
            parsed_url = urlparse(image_url)
            path_segments = [s for s in parsed_url.path.split('/') if s]

            suggested_name = ""
            if path_segments:
                if path_segments[-1].lower() == 'manifest' and len(path_segm
                    suggested_name = path_segments[-2]
                else:
                    suggested_name = path_segments[-1]

            suggested_name = re.sub(r'[^\w\-\.]', '', suggested_name).strip(
            url_short_hash = hashlib.md5(image_url.encode('utf-8')).hexdiges

            if '.' in parsed_url.path:
                potential_ext = os.path.splitext(parsed_url.path)[1].lower()
                if potential_ext in ['.jpg', '.jpeg', '.png', '.gif', '.bmp'
                    ext = potential_ext

            # This is the initial filename for download
            if suggested_name:
                initial_download_filename = f"{suggested_name}_{url_short_ha
            else:
                initial_download_filename = f"{url_short_hash}{ext}"

            if len(initial_download_filename) > 200:
                initial_download_filename = f"{url_short_hash}{ext}"

            generated_filename = initial_download_filename # Store for loggi
            image_path = os.path.join(output_folder, initial_download_filena

            if os.path.exists(image_path):
                status = "Skipped (Exists)"
                skipped_count += 1
                # Log uses the existing 'generated_filename' (initial_downlo
```

```
                download_log.append({
                    'dataframe_index': index,
                    'original_url': image_url,
                    'generated_filename': generated_filename,
                    'download_status': status,
                    'error_message': None
                })
                continue

            print(f"Downloading image for URL index {index}: '{image_url}' a
            response = requests.get(image_url, stream=True, timeout=15)
            response.raise_for_status()

            with open(image_path, 'wb') as f:
                for chunk in response.iter_content(chunk_size=8192):
                    f.write(chunk)

            # --- Successful Download: Rename to image_X format ---
            image_sequential_counter += 1
            final_image_name = f"image_{image_sequential_counter}{ext}" # Ch
            final_image_full_path = os.path.join(output_folder, final_image_

            original_downloaded_path = image_path # Path with initial_downlo

            try:
                os.rename(original_downloaded_path, final_image_full_path)
                print(f"Successfully downloaded. Renamed '{initial_download_
                generated_filename = final_image_name # Update for logging t
            except OSError as e_rename:
                print(f"Error renaming '{initial_download_filename}' to '{fi
                # generated_filename remains initial_download_filename
                if error_message:
                    error_message += f"; Rename to {final_image_name} failed
                else:
                    error_message = f"Rename to {final_image_name} failed: {

            status = "Downloaded"
            download_count += 1
            time.sleep(0.1)

        except requests.exceptions.RequestException as req_err:
            status = "Error"
            error_count += 1
            print(f"Error downloading '{image_url}': {req_err}")
            error_message = str(req_err)
        except Exception as e:
            status = "Error"
            error_count += 1
            print(f"An unexpected error occurred for '{image_url}': {e}")
            error_message = str(e)

        download_log.append({
            'dataframe_index': index,
            'original_url': image_url,
            'generated_filename': generated_filename, # This will be image_X
            'download_status': status,
```

```
            'error_message': error_message
        })

    print("\n--- Download Summary ---")
    print(f"Total images downloaded and named/renamed: {download_count}")
    print(f"Total images skipped (due to existing files or invalid URLs): {s
    print(f"Total images failed (due to download errors): {error_count}")
    print(f"Images saved in: '{os.path.abspath(output_folder)}'")

    # --- 4. Save the Download Log to CSV ---
    log_df = pd.DataFrame(download_log)
    log_filepath = os.path.join(output_folder, "image_download_log.csv")
    try:
        log_df.to_csv(log_filepath, index=False, encoding='utf-8')
        print(f"\nDownload log saved to: '{log_filepath}'")
    except Exception as e:
        print(f"Error saving download log: {e}")

    return output_folder


def zip_folder(folder_path, zip_name):
    """
    Compresses a specified folder into a zip archive.
    """
    try:
        with zipfile.ZipFile(zip_name, 'w', zipfile.ZIP_DEFLATED) as zipf:
            for root, dirs, files_in_dir in os.walk(folder_path):
                for file_item in files_in_dir:
                    file_path = os.path.join(root, file_item)
                    zipf.write(file_path, os.path.relpath(file_path, folder_
        print(f"\nSuccessfully created zip archive: '{os.path.abspath(zip_na
    except Exception as e:
        print(f"Error creating zip archive for '{folder_path}': {e}")


# --- Script Execution ---
if __name__ == "__main__":
    # --- Configuration ---
    # IMPORTANT: Ensure your file is accessible (upload to Colab or mount Go
    your_dataframe_file = 'DatasetCleanedEuropeanaTeapotsExcel.csv' # <--- C
    your_image_url_column_name = 'ImagePreview' # <--- CHANGE THIS TO YOUR I
    your_output_folder_name = 'downloaded_sequential_images' # Changed folde
    your_zip_file_name = f"{your_output_folder_name}.zip"

    # 1. Run the download process
    downloaded_folder = download_images_from_dataframe(
        dataframe_path=your_dataframe_file,
        image_url_column=your_image_url_column_name,
        output_folder=your_output_folder_name
    )

    # 2. After downloading, zip the folder
    if downloaded_folder:
        zip_folder(downloaded_folder, your_zip_file_name)
```

```
        # 3. Trigger the download of the created zip file in Colab
        try:
            print(f"\nAttempting to download '{your_zip_file_name}' (Colab s
            files.download(your_zip_file_name)
        except NameError: # Handles if 'files' from google.colab is not defi
            print(f"\n'files.download()' is a Colab specific function. If nc
        except Exception as e:
            print(f"Error during Colab files.download: {e}")
```

The problem was that the images had a unique title name, but it was not retraceable to the records of the teapots themselves. There was no logical way to find which images belongs to which information of our dataset. This was problematic. I asked to simply name the images to 'image 1' to 'image x'. I integrated this in the code above.

4.1.2.1 IIIF Manifest URL's: High Quality

The thumbnail/preview was not completely satisfactory, so I looked in how to visualize the high quality images. This can be done via iiif API's of Europeana. If I first get the iiif manifest urls, I can use them to visualise the images witht the help of the Mirador or Universal Viewer. To do this I needed to develop a html script, so I could integrate these viewers (and manifests) onto a local of web-based server (a website).

To do this, I first scripted a way to get all the iiif manifest urls from the dataset we created from chapter one to chapter three. I took my final script from chapter one and adjust it with the help of Gemini to download the Europeana id and iiif url and save them in a separate document (csv). I open it in Openrefine and crossreference it with my original dataset I ended up with at the end of chapter three to get the iiif manifests for all 3582 records in my cleaned dataset.

In [ ]:
```
# Script to fetch Europeana ID and IIIF URL with Record API validation
# generated by Gemini 2.0 on 15/04/2025
# authorized by Jenske Verhamme
# Validation step re-added on 02/05/2025

# Import necessary libraries
import requests  # Used for making HTTP requests
import csv  # Used for writing data to CSV files
import os  # Used for creating directories and handling file paths
import time  # Used for adding delays between requests
from urllib.parse import quote_plus  # Used for encoding special characters
try:
    from google.colab import files  # Used for downloading files from Google
    IN_COLAB = True
except ImportError:
    IN_COLAB = False # Not running in Colab
import json  # Used for handling JSON data
```

```python
# <<< --- PASTE YOUR ACTUAL EUROPEANA API KEY HERE --- >>>
API_KEY = "reeditaccif"

# Define a function to send requests with retries (same as before)
def send_request(url, retries=3, backoff_factor=1.5): # Reduced retries slig
    attempt = 0
    response = None  # Initialize response outside the try block
    while attempt < retries:
        try:
            # Add a timeout to the request to prevent hanging indefinitely
            response = requests.get(url, timeout=15) # 15 second timeout
            response.raise_for_status()
            # Check for explicit API error messages even on 200 OK for Recor
            if "/record/" in url: # Only apply this check to record API call
                try:
                    data = response.json()
                    if not data.get('success', True):
                        print(f"Record API returned error: {data.get('error'
                        return None # Treat API error as failure
                except json.JSONDecodeError:
                    print(f"Record API response not valid JSON: {response.te
                    return None # Treat invalid JSON as failure
            return response
        except requests.exceptions.Timeout:
            print(f"Attempt {attempt + 1} timed out after 15 seconds.")
            attempt += 1
        except requests.exceptions.RequestException as e:
            # Be less verbose for 404 errors during validation as they are e
            if isinstance(e, requests.exceptions.HTTPError) and e.response.s
                # print(f"Validation failed: Record not found (404) for {url
                return None # Treat 404 as validation failure immediately
            else:
                print(f"Attempt {attempt + 1} failed: {e}")
                attempt += 1

        if attempt < retries and response is None: # Check if response is st
            wait_time = backoff_factor ** attempt
            # print(f"Retrying in {wait_time:.2f} seconds...") # Less verbos
            time.sleep(wait_time)

    # print("Max retries reached or validation failed.") # Less verbose fail
    return response

# Main function to fetch IDs and construct IIIF Manifest URLs with validatio
def fetch_europeana_manifests_validated(api_key, queries, rows=100, limit=10
    directory = f'downloads_manifests_validated/{int(time.time())}/'  # New
    os.makedirs(directory, exist_ok=True)

    validated_records_count = 0 # Count records that pass validation
    processed_search_results = 0 # Count total search results processed
    cursor = '*'
    all_manifest_data = [] # List to store dicts of {'europeana_id': id, 'ii
    csv_file_path = os.path.join(directory, 'europeana_manifest_urls_validat

    fieldnames = ['europeana_id', 'iiif_manifest_url']
```

```
    combined_query = ' OR '.join(queries)

    # Estimate total records (optional) - based on search index, not validat
    estimated_total_records = None
    search_url_estimate = f'https://api.europeana.eu/api/v2/search.json?wske
    estimate_response = send_request(search_url_estimate, retries=2) # Lower
    if estimate_response:
        try:
            estimate_data = estimate_response.json()
            estimated_total_records = estimate_data.get('totalResults')
            print(f"Estimated total search results for '{combined_query}': {
        except json.JSONDecodeError:
            print("Could not estimate total results.")

    print(f"Starting fetch for query: {combined_query}. Aiming for up to {li


    # Fetch records loop - continues until limit of *validated* records is m
    while validated_records_count < limit:
        # Fetch a batch of search results
        search_url = f'https://api.europeana.eu/api/v2/search.json?wskey={ap
        print(f"\nFetching search batch using cursor: {cursor}")

        search_response = send_request(search_url, retries=5, backoff_factor
        if search_response is None:
            print("Failed to fetch search batch, stopping.")
            break

        try:
            search_data = search_response.json()
        except json.JSONDecodeError as e:
            print(f"JSON decode error for search batch: {e}")
            print(f"Response text: {search_response.text[:200]}")
            break # Stop if search results are invalid

        if not search_data.get('success', True):
            print(f"Search API Error: {search_data.get('error', 'Unknown err
            break

        items = search_data.get('items')
        if not items:
            print("No more items found in search results.")
            break

        print(f"Processing {len(items)} items from search batch...")
        items_validated_in_batch = 0

        # Loop through each item found in the search batch
        for item in items:
            if validated_records_count >= limit:
                print("Target limit reached.")
                break # Stop processing this batch if limit hit

            processed_search_results += 1
            record_id = item.get('id')
```

```
            if record_id and isinstance(record_id, str):
                cleaned_record_id_for_url = record_id.lstrip('/')

                # --- !!! Add Validation Step !!! ---
                record_url = f'https://api.europeana.eu/api/v2/record/{clear
                record_response = send_request(record_url) # Uses lower retr

                # --- Only proceed if the Record API call was successful ---
                if record_response:
                    # Construct IIIF Manifest URL
                    iiif_manifest_url = f"https://iiif.europeana.eu/presenta

                    manifest_data = {
                        'europeana_id': record_id,
                        'iiif_manifest_url': iiif_manifest_url
                    }
                    all_manifest_data.append(manifest_data)
                    validated_records_count += 1 # Increment count only for
                    items_validated_in_batch += 1

                    # Print progress periodically
                    if validated_records_count % 50 == 0 or validated_record
                        print(f"Validated record {validated_records_count}/{

                # else: # Optional: Log skipped items
                #     # print(f"Skipped item {record_id} - Failed Record API
                #     pass

                # --- Add a small delay within the loop to avoid overwhelmin
                # Adjust sleep time as needed. Start small. Remove if not ne
                time.sleep(0.05) # 50 milliseconds delay between record chec

            else:
                print(f"Skipping item due to missing/invalid ID in search re

        # --- End of loop for items in batch ---
        print(f"Validated {items_validated_in_batch} items in this batch.")

        if validated_records_count >= limit:
            print("Target limit reached after processing batch.")
            break # Exit outer loop if limit hit

        if 'nextCursor' in search_data:
            cursor = search_data['nextCursor']
        else:
            print("No nextCursor found in search results. Assuming end.")
            break # Exit outer loop if no more search results

        # Optional: slightly longer sleep between batches
        # time.sleep(0.5)

    # --- End of while validated_records_count < limit loop ---

    print(f"\nWriting {len(all_manifest_data)} validated ID/Manifest URL pai
    print(f"(Processed {processed_search_results} total search results to fi
```

```
    try:
        with open(csv_file_path, mode='w', newline='', encoding='utf-8') as
            writer = csv.DictWriter(file, fieldnames=fieldnames)
            writer.writeheader()
            writer.writerows(all_manifest_data)

        print(f"Process completed. {len(all_manifest_data)} validated record
        print(f"CSV file saved at: {csv_file_path}")

        if IN_COLAB:
            print("Attempting to download file in Colab...")
            files.download(csv_file_path)
        else:
            print(f"File saved locally at: {csv_file_path}")

    except IOError as e:
        print(f"Error writing CSV file: {e}")
    except Exception as e:
        print(f"An unexpected error occurred during CSV writing: {e}")

    return all_manifest_data # Return the list of validated data


# --- Define search terms (same as before) ---
teapot_translations = {
    'en': ['teapot', 'tea?pot'],
    'nl': ['theepot', 'tee?pot'],
    'de': ['teekanne', '?eekanne'],
    'fr': ['théière', 'th?i?re', 'bouilloire'],
    'es': ['tetera'],
    'it': ['teiera'],
    'sl': ['čajnik'],
    'sv': ['teekanne'],
    'ca': ['tetera'],
    'pt': ['teiera', 'te?iera'],
    'ro': ['teiera'],
    'lt': ['teiera'],
    'uk': ['чайник'],
    'no': ['kettle'],
    'sr': ['čajnik', '?ajnik'],
    'fi': ['teekanne', 'te?kanne'],
    'da': ['teiere', 't?eiere'],
    'is': ['kettle'],
    'lv': ['teiera'],
    'pl': ['czajnik', 'cajnik'],
}
queries = list(set([term for terms in teapot_translations.values() for term
print(f"Using {len(queries)} unique search terms.")

# --- Set desired limit for VALIDATED records ---
# This should now result in a count closer to your original ~4500
desired_limit = 15000 # Aim for up to 15000 validated records

# --- Run the validated function and store the result ---
if API_KEY == "YourapiKey":
    print("\n--- WARNING: Please replace 'reeditaccif' with your actual Euro
```

```
else:
    validated_data = fetch_europeana_manifests_validated(API_KEY, queries, l

# --- Print statement of estimated results ---
# The estimated total search results are printed within the `fetch_europeana
# Now we can access the returned `validated_data` list.
print(f"\n--- Estimated and Final Results ---")
# The estimated total number of records based on the initial search query is
# The final number of validated records is now accessed from the returned li
if 'estimated_total_records' in locals() and estimated_total_records is not
    print(f"Estimated total matching records (before validation): {estimated
else:
    print("Estimated total matching records (before validation): Could not b

# The actual number of validated records is the length of the returned list.
if 'validated_data' in locals():
    print(f"Number of validated records with IIIF Manifest URLs: {len(valida
else:
    print("Number of validated records with IIIF Manifest URLs: Could not be
```

Next I matched this database with 8465 records to my original dataset and was able to retrieve the manifest for my 3542 records. I added them to my original dataset in Openrefine. To do this I opened my original dataset and the dataset above. I made sure to name the Europeana id to EID for both projects. Then I created a new column on the basis of the EID column in my original project and linked the iiif url's from project B. This is the same process as we did in chapter two with cross reference.

code: with(cell.cross("all_manifest_data", "EID"), matches, if(matches.length() > 0, matches[0].cells["IIIFmanifesturl"].value, "no match"))

After the crossreference we have our original dataset with added IIIF manifest urls. These urls can be used in Mirador or Universal viewer to get the image and metadata of a record.

remark: these urls might need to be cleaned up as they may still contain your Europeana API key. This can easily be done with the replace function in OpenRefine.

4.1.2.2 HTML: Mirador

Next I wanted to write a html script to look at the images behind the iiif manifests. I ask Gemini to help me write a basic script to load in some iiif urls into mirador and show it as a viewer.

In [ ]:

```
<!DOCTYPE html>
<html>
<head>
  <title>Mirador Viewer</title>
```

```
  <script src="https://unpkg.com/mirador@latest/dist/mirador.min.js"></scrip
</head>
<body>
  <div id="viewer" style="width: 100%; height: 800px;"></div>
  <script>
    const config = {
      id: "viewer",
      windows: [
        {
          manifestId: "https://iiif.europeana.eu/presentation/815/item_32806
        },
        {
          manifestId: "https://iiif.europeana.eu/presentation/2048053/MUO_03
        },
        {
          manifestId: "https://iiif.europeana.eu/presentation/2064930/https_
        }
      ]
    };

    Mirador.viewer(config);
  </script>
</body>
</html>
```

This code shows the results for 3 iiif manifests. To run this I copy the code that gemini produces into notepad and save it as 'MiradorViewer.html', and make sure to click the 'all file types/all files' below it to not save it as a text, but as a html document. If I now click on the document in my document folder, it wil open the script in my browser and I will see the mirador viewer with three images related to teapots.

Next I wanted to add strategy to load up all the urls of my iif images. If I added more, for example 90. This would mean I open up 90 canvasses at the same time. This seemed to be unwanted and overload the browser/system. I had to find another solution to solve this.

First I tried a strategy to copy the html script into the notepad to create a html. I did this by saving the document with the name 'MiradorViewer.html' and clicked 'all file types' beneath the name. If we click on this file under documents and run it, it wil open in the browser and show the Mirador interface with 3 ifff manifests. Yet, I could not load many images and they would all open as canvasses. I could also not load in an entire file via json, whatever I tried.

next I thought it was because i need to host a server, because a file cannot be accesed via browser testing. I found a strategy to first open the windows command prompt and create a folder 'MiradorProject' and put the script in it. I then ran a python server in the folder. how to do this: first open the command prompt, go to folder by pressing 'cd' and then the file location 'C:\Users\Jenske

Sampson\Downloads\mirador-project', then press enter to go to that location. Next you can start the server (if python is installed properly). to do this you can enter 'python -m http.server 8000' and it should start a server at http://localhost:8000. To acces the server one could go that link.

I succesfully made a server but was not able to load properly. Even when I put the script in a new folder 'MiradorProject' and put the scriptfile 'MiradorViewer' into it. And also the Json to which it referred. To make the Json I extracted the iiif urls from my OpenRefine file via export 'custom tabular' and choose the 'iiif manifests' column. I exported it as csv and used a online converter (https://www.convertcsv.com/csv-to-json.htm) to convert the document to an JSON array.

I saved this document and also put it in the 'MiradorProject'. This should be used by the new script to load the images. This partly worked but also crashes the system, because it opens all urls at the same time as new canvasses (all 3500+!). I had to find another solution. I did, partly.

I went back to the original script and pasted all links of the json manually in the file. This worked but also crashed constantly. Mirador was not suited to do this. I did notice you can add one url at a time. I wanted to find a way to upload all urls at the same time. That is how I came to the solution to create my own IIIF manifest that serves as a collection for all these records.

4.1.2.3 IIIF collection manifest

I thought I could maybe combine all the urls under one new link. I looked further into this matter **and** came onto the idea of a iiif collection manifest url. this combines all the manifest urls into one collection url.

https://pro.europeana.eu/discover-the-data/apis
https://pro.europeana.eu/post/building-a-rich-media-experience-with-the-europeana-api-and-iiif

The code below shows how to make a collection manifest iiif. this file contains all the diffrent iiif manifest urls and combines them into one file and a separate link to load all the records in one go. This is important because otherwise you can only load in one manifest or url at a time in Mirador. As i do not want to load in +- 3500 records by hand, I tried to construct them via a script.

another problem was that i still have to add the iiif manifest urls to the script below. To do this I had to extract these urls from Openrefine. There I exported the column with the urls as a csv, and converted it online to a Json. That list got copied below here into the script. This was tedious. This ended up with code that

has 3500+ urls in it. That is not presentable. Therefore after consulting Gemini, I came up with a solution to take the extracted Json dataset and I would copy paste all these urls and place them in a .txt file. Then upload the txt here into Google Colab. This document can then be consulted and be used by the script below to create the iiif collection manifest url from the iiif manifest urls without having thousands of lines of code.

In [ ]:
```
# load the txt extraction of the manifest urls (from Openrefine) here.
from google.colab import files
uploaded = files.upload()
```

In [ ]:
```
#code to change iiif manifest urls to a json that can serve as a iiif collec
import json
import requests # Required only for fetching labels automatically
import time     # Used for a small delay when fetching

# --- Configuration ---
COLLECTION_ID = "https://your-username.github.io/my-iiif-data/collection.jsc
COLLECTION_LABEL = "My Awesome IIIF Collection" # Replace with your Collecti
OUTPUT_FILENAME = "my-collection.json" # Name of the output file
FETCH_LABELS = True # Set to False if you don't want to fetch labels automat
REQUEST_TIMEOUT = 10 # Seconds to wait for each manifest request
REQUEST_DELAY = 0.1 # Seconds to wait between requests (politeness)

# List of manifest URLs (replace with your actual URLs)
# Option 1: Define list directly in the script
#manifest_urls = [

#]
# Option 2: Read from a file (uncomment the following lines if using this)
manifest_urls = []
try:
      with open("iiif_manifest_urls.txt", "r") as f:
         manifest_urls = [line.strip() for line in f if line.strip()]
except FileNotFoundError:
     print("Error: iiif_manifest_urls.txt not found.")
     exit()

# --- Script Logic ---
collection = {
    "@context": "http://iiif.io/api/presentation/3/context.json",
    "id": COLLECTION_ID,
    "type": "Collection",
    "label": {"en": [COLLECTION_LABEL]}, # Assuming English label
    "items": []
}

print(f"Processing {len(manifest_urls)} manifests...")

for index, url in enumerate(manifest_urls):
    print(f"[{index+1}/{len(manifest_urls)}] Processing: {url}")
    item = {
        "id": url,
```

```
        "type": "Manifest"
        # Label will be added below if fetched
    }

    if FETCH_LABELS:
        manifest_label = f"Manifest {index+1}" # Default label
        lang_code = "en" # Default language

        try:
            # Add headers to potentially look more like a browser
            headers = {'User-Agent': 'Python IIIF Collection Script/1.0'}
            response = requests.get(url, timeout=REQUEST_TIMEOUT, headers=he
            response.raise_for_status() # Raise an error for bad status code
            manifest_data = response.json()

            # Try to get label (works for V3 and most V2)
            if 'label' in manifest_data and manifest_data['label']:
                label_obj = manifest_data['label']
                # Handle V3 structure (language map)
                if isinstance(label_obj, dict):
                    # Try common language codes first
                    if 'en' in label_obj and label_obj['en']:
                        manifest_label = label_obj['en'][0]
                        lang_code = 'en'
                    elif 'none' in label_obj and label_obj['none']:
                         manifest_label = label_obj['none'][0]
                         lang_code = 'none'
                    else:
                        # Grab the first available language/value pair
                        first_lang = next(iter(label_obj))
                        if label_obj[first_lang]:
                            manifest_label = label_obj[first_lang][0]
                            lang_code = first_lang
                # Handle simple V2 string label
                elif isinstance(label_obj, str):
                    manifest_label = label_obj
                    lang_code = 'en' # Assume English or unspecified

            print(f"  -> Fetched label: '{manifest_label}' ({lang_code})")
            item["label"] = {lang_code: [manifest_label]} # Add label to ite

        except requests.exceptions.RequestException as e:
            print(f"  -> Error fetching manifest: {e}")
            item["label"] = {"en": [f"Manifest (Error Fetching: {url})"]} #
        except json.JSONDecodeError:
            print(f"  -> Error decoding JSON from manifest.")
            item["label"] = {"en": [f"Manifest (Invalid JSON: {url})"]}
        except Exception as e:
            print(f"  -> An unexpected error occurred: {e}")
            item["label"] = {"en": [f"Manifest (Processing Error: {url})"]}

        time.sleep(REQUEST_DELAY) # Be polite to servers

    else:
         # If not fetching, add a generic label
         item["label"] = {"en": [f"Manifest {index+1}"]}
```

```
    collection["items"].append(item)

# --- Write Output ---
try:
    with open(OUTPUT_FILENAME, "w", encoding="utf-8") as f:
        json.dump(collection, f, ensure_ascii=False, indent=2) # Use indent
    print(f"\nSuccessfully created collection manifest: {OUTPUT_FILENAME}")
except IOError as e:
    print(f"\nError writing file {OUTPUT_FILENAME}: {e}")
```

In [ ]:
```
from google.colab import files

files.download('my-collection.json')
```

With the script above I was able to create a JSON structure that could serve as a iiif manifest on its own. It extracts the iiif manifest url of every record in our dataset. However, I cannot upload every record manually on Mirador. Therefore I will need to look for a solution to upload all iiif manifests (urls) at once into Mirador. I first uploaded the result of the script on github to create a collection iiif manifest url. I could not make a html script work with the complete list. I thought this was possible via scripting, but did not find a solution yet. I was able to do it differently though.

how to make it a url?

I made a new repository on Github and called it Mirador Project. in that folder i uploaded the adjusted json above. When I push it and upload it publicly, it has a url that can serve as a iiif manifest url (of a collection of iiif manifest urls).

in this case: https://guacamolekoala.github.io/iiif-test/my-collection.json

with label added as title coul be an option. But actually, I like the naming in Mirador with 'Manifest xx' as an unique identifier, as you can also see the title when you click on the item.

Collection manifest into Mirador

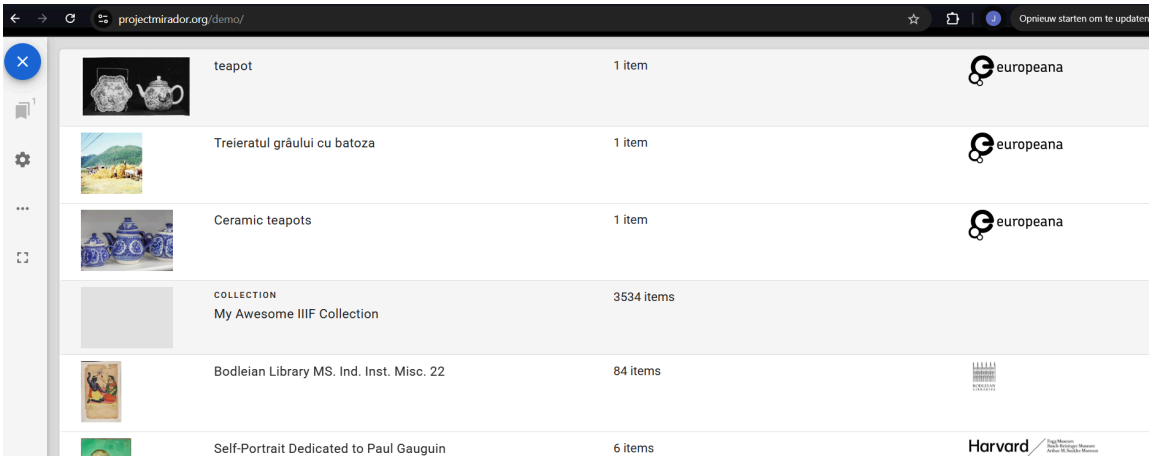
If we open the demo version of Mirador:

link: https://projectmirador.org/demo/

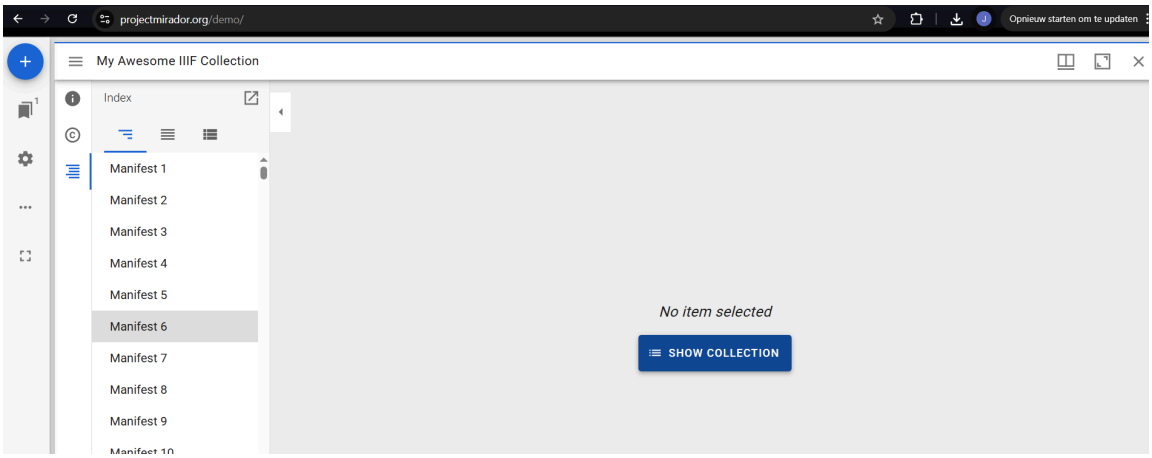
Here we can click on the + sign on the left. After clicking we can add a resource. Here we can add the collection url that we created on GITHUB.

link: https://guacamolekoala.github.io/iiif-test/my-collection.json

if we add this URL we see a loaded set with around 3500 items.



If we click on the collection (in this case the My Awesome IIIF Collection) it will load in on Mirador. Next we can select between the different manifests to see the respective images and metadata of records in our dataset.



If we then click on 'show collection' and select an item at random in the list of manifests we will get a view of an image. If on the right we click on 'toggle sizebar' (the three horizontal lines) and then click on the index (the 5 horizontal lines a bit lower). We can select the collection name (in this case my awesome iiif collection). If we click on this, the collection overview bar on the right will show up. Here we can easily look at all the records individually, but in one neat interface.

remark: to set the black view, you can just select the theme in the menu bar on the left via the settings icon.

Now we can look at all the images via the Mirador viewer.

However, I was not able to solve the problem of loading Mirador with the collection via html webserver. This means that our dataset can only be properly consulted via the Mirador demo, instead of via local or web based server. I hope to solve this problem in the future.

I was able to write a html-script via Gemini that loads in the collection into a mirador viewer and can be tested via a local or webbased server. However, when we click on 'start here' in the Mirador viewer and then click on the 'awesome collection', we get stuck in a white screen.

After a long search, I concluded that the set might be too big when using a browser or a local server. This did not seem to be a problem with the demo. I tried creating subsets with 500, 100 and only 5 records. Created new collection manifest urls, but nothing seemed to work.

I found the solution! Solving the problem in Gemini: After prompting for ages I was able to find a solution. This was the case because I was able to find a new method of coding with Gemini. I kept rephrasing the problem to Gemini, but whatever I did I could not fix the white screen.

Suddenly I noticed that I can preview scripts inside of Gemini via the canvas option. Here I can choose between code view and an example view of the code. When I was clicking in the preview of this code and got to the white screen again (this time via the previewer of Gemini), it gave an error with the option to fix the code on the spot by Gemini. I choose this option and the problem got solved.

In [ ]:
```
# HTML Script to use the mirador viewer with our collection manifest
# Code generated by Gemini 2,5
```

```
# Authorized by Jenske Verhamme

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Teapot Collection (Mirador)</title>
    <link rel="stylesheet" href="https://unpkg.com/mirador@3.4.0/dist/mirado
    <style>
        /* Make Mirador fill the entire page */
        html, body, #mirador-viewer {
            margin: 0;
            padding: 0;
            height: 100%;
            width: 100%;
            overflow: hidden; /* Prevent scrollbars */
        }
    </style>
</head>
<body>
    <div id="mirador-viewer"></div>

    <script src="https://unpkg.com/mirador@3.4.0/dist/mirador.min.js"></scri

    <script type="text/javascript">
        // --- YOUR IIIF COLLECTION MANIFEST URL ---
        // IMPORTANT: This URL should point to the manifest served by your l
        // It assumes you have run the Python script and are serving the 'ou
        // If you are using the original 'my-small-collection.json' directly
        // ensure GitHub Pages is enabled for your repository and use the co
        const MY_IIIF_COLLECTION_MANIFEST_URL = 'https://raw.githubuserconte

        // Initialize Mirador viewer
        Mirador.viewer({
            id: 'mirador-viewer', // ID of the HTML element where Mirador wi

            // Add your collection to Mirador's catalog. This is how Mirador
            catalog: [{
                manifestId: MY_IIIF_COLLECTION_MANIFEST_URL,
                provider: 'My Teapot Collection' // Custom label for your co
            }],
            // Workspace settings for collections
            workspace: {
                type: 'mosaic', // 'mosaic' allows multiple windows; 'single
            },
            window: {
                // For collections, starting with the table of contents is u
                defaultSidebarPanel: 'tableOfContents',
                sideBarOpenByDefault: true,
                panels: {
                    info: true,
                    toc: true, // Table of Contents panel is crucial for col
                    annotations: true,
                    search: true
                }
```

```
            },
            // Enable debugging to see Mirador's internal logs in the browse
            // This is CRUCIAL for identifying any issues with your manifest
            debug: true
        });
    </script>
</body>
</html>
```

After downloading the script as .html and running it, it will use the collection url and open up Mirador. In Mirador you can then click 'start here' or '+'. In the next screen our collection will be found and will be selectable.

4.1.3 Universal Viewer

I also tried a htmls script with universal viewer. I ran into similar problems as with Mirador. I reached a white screen upon uploading the dataset and running the script. Because I found a working solution in Mirador, I decided to not use Universal Viewer.

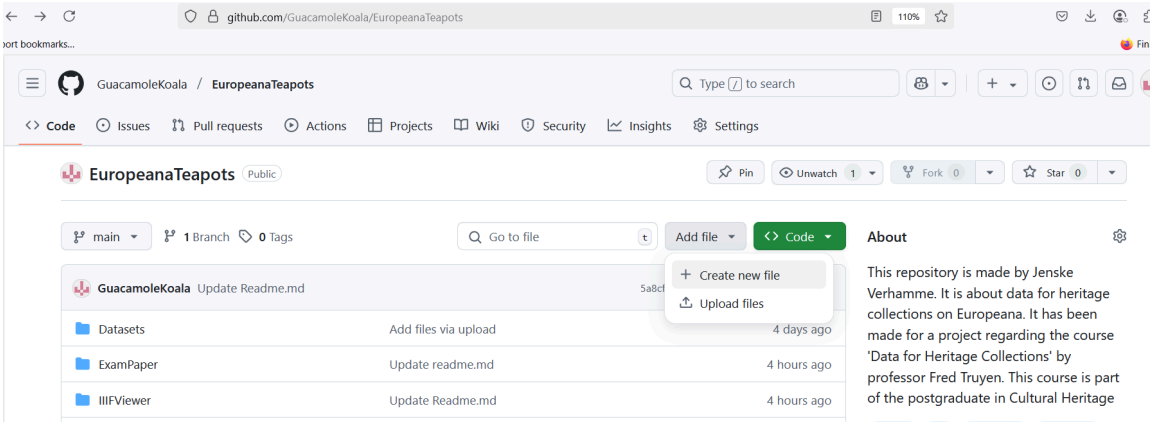
4.2 GITHUB

In this part I built a Github repository to share my project and have a version control available.

how to make a repository?

First I made an account and logged in to Github. There I clicked on 'create new' and choose 'repository'.

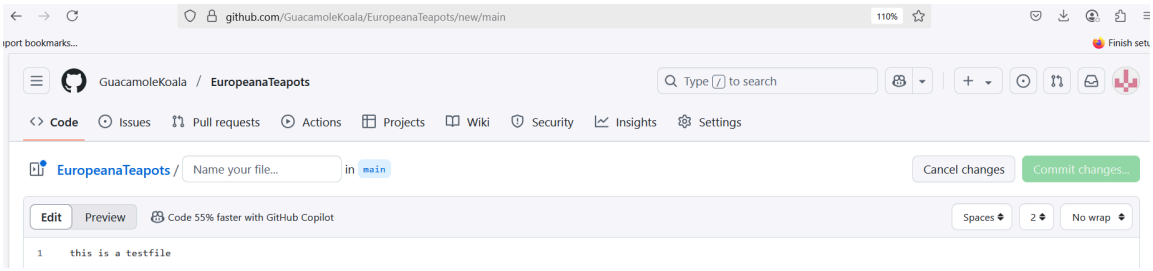
In this 'folder'/repository we can create several folders for several aspects of our project. Here I give an overview of what folders and files are there. To make a folder you can just add a file to the repository and then click 'create a file'.



After creating new file, you get a screen where you can choose your file name. If you enter a name and enter a '/' behind it, it becomes a new folder.

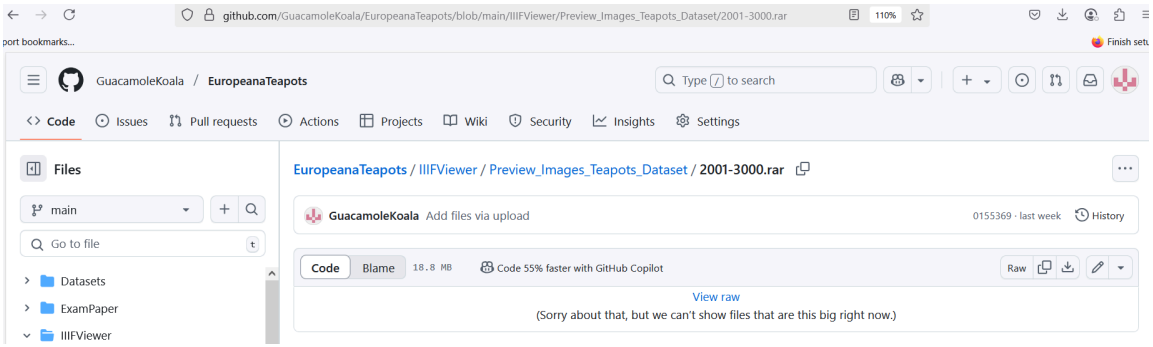
example: if we start from EuropeanaTeapots and create a new file, we can enter 'Datasets/' to create a new folder called 'datasets', we can then enter a new file name after the slash. Afterwards you can upload files into this new folder.

This way we can make many folders and create a hierarchy in our repository. Below you can see the place where you can enter the filename.



Below you can see the principle in action by creating a folder in a folder. In EuropeanaTeapots, we create a folder 'IIIFViewer' and in that folder we create another one, namely 'Preview_images_teapots_dataset', and in this folder we place some files, such as images 1-1000, 1001-2000, and so on.

link: https://github.com/GuacamoleKoala/EuropeanaTeapots/tree/main/IIIFViewer/Preview_



Overview of files

name of repository: EuropeanaTeapots link: https://github.com/GuacamoleKoala/EuropeanaTeapots

- folder 'Datasets': here you can find all the datasets regarding the project. These are 1) an initial dataset, 2) an extra dataset, 3) the cleaned dataset in OpenRefine and 4) the cleaner dataset in Tableau.
- folder 'IIIFViewer': here you can find all files regarding the part about 'IIIF' viewers. Here you can find: 1) A folder with all thumbnail images and a

logbook, 2) a .txt with all IIIF manifest urls of our dataset, 3) a json file that serves as a IIIF collection manifest url for the dataset, 4) a html script to watch the dataset in Mirador viewer.

- folder Manual: here you can find all steps that were made during the project. The manual is divided into four chapters: 1) API and Python, 2) OpenRefine, 3) Excel/Tableau, 4) HTML, IIIF, Mirador. These files are provided as python notebook and as pdf.

- folder Visualisations: here you can find all the visualisations that are made in Tableau. You can find 1) the Tableau workbook and 2) the pdf versions here.

- folder Examfile: Here you find the examfile that is the culmination of this project. This project and exam are for a course in the postgraduat in Cultural Heritage.

- readme: for every folder I tried to make a readme that explains what files are in the folder and what to do with them. A good tip is to create the readmes, not as .txt, but as .md in the repository. This shows them visually in the repository without a preview.